AI 2014: Web Technology

SYAI&DS

Web Technology : AI 2014

- Course Prerequisites: Computer Programming, Database Management Systems, Computer Network

Syllabus- SECTION-1

- Introduction: Introduction to web technology, Internet and WWW, web site planning and design issues, HTML5: structure of html document, commenting, formatting tags, list tags, hyperlink tags, image, table tags, frame tags, form tags, CSS, Bootstrap, JSON(6hrs)

- Client Side Technologies: JavaScript: Overview of JavaScript, Data types, Control Structures, Arrays, Functions and Scopes, Objects in JS, Form validation, DOM: Introduction, DOM levels, DOM Objects, their properties and methods, Manipulating DOM (6 hrs)

- JQuery: Introduction, Loading JQuery, selecting elements, changing styles, creating elements, appending elements, removing elements, handling events.(2 hrs)

Syllabus- SECTION-2

- ServerSide Technologies: Introduction, Features, PHP syntax, Control structures, Functions, Arrays, String,Form Handling, File Handling, Session and Cookies, Error Handling, MySQL with PHP (6 Hrs)

- React: Introduction,Architecture, Components, JSX, Class, State, Props, Events, Render function, Forms, Lists, Router, Flux. (4 Hrs)

- NodeJS: Introduction, Installation of Node JS, Node JS Modules, Node Package Manager (npm), Creating Web server, File System, Express JS, Serving Static Resources, Database connectivity. (4 Hrs)

Assessment Scheme

Sr. No	Component	Marks
1.	ESE (Online MCQ)	60
2.	Lab Assessment	100
3.	Course Project	100
4.	CVV	100
5.	Home Assignment	100

Introduction to Web Technology

- Web technology refers to the various tools and technologies that are utilized in the process of communication between different types of devices over the Internet.

Internet

- A short form of INTERconnected NETwork.
- A giant network of networks
- Includes PCs, and other devices like servers, printers
- Connected through a communication channel
- 1962: ARPANET (Advanced Research Project Agency Network)

Internet

- Ethernet developed by Xerox
- 1973: TCP/IP developed by Vint Cert and Bob Kahn.
- What is the use?

Email, Gaming, Video Conferencing, Streaming Audio, Video Blogging, Software Upload/ Download etc..

WWW

- Set of technologies that allowed information on the internet to be linked together.
- Developed in 1989 by Tim Berners-Lee.
- Related terms:

1. IP address (eg: 192.168.11.66)
2. Domain Names (eg: vit.edu)
3. URL (eg: https://www.vit.edu)
4. HTTP
5. Client Server Computing
6. Intranet and extranet

Tools Required for Web Site Designing

- PC/Notebook/Laptop
- Text editor

  Sublime text, Brackets, Atom, Visual Studio Code, Notepad, gEdit, etc..
- Web Browser

  Firefox, Chrome, Safari, opera, internet Explorer, Microsoft Edge, etc..
- Graphics editor

  Adobe Photoshop, Paint, etc..
- Version Control System

  Git

Web Site Design Issues

- Browser and Operating Systems
- Bandwidth and Cache
- Display resolution
- Look and feel

1. Website theme
2. Fonts, Graphics and colors
3. Presentations and Access

Web Site Design Issues

- Page Layout and linking

  Grid layouts and Navigation links should be used

- Locating Information

  Put most important information in the center then top, right, bottom, left

- Make Design User-Centric

  Publish content within column, provide table of contents as navigations

- Sitemap

  Organize content hierarchically and provide links of related pages.

Website Planning

- Objectives and Goals
- Audience/ User Profile
- Sketching out your design

1. Take a pen and paper and try some rough sketches of your website
2. Mockup tools are also available

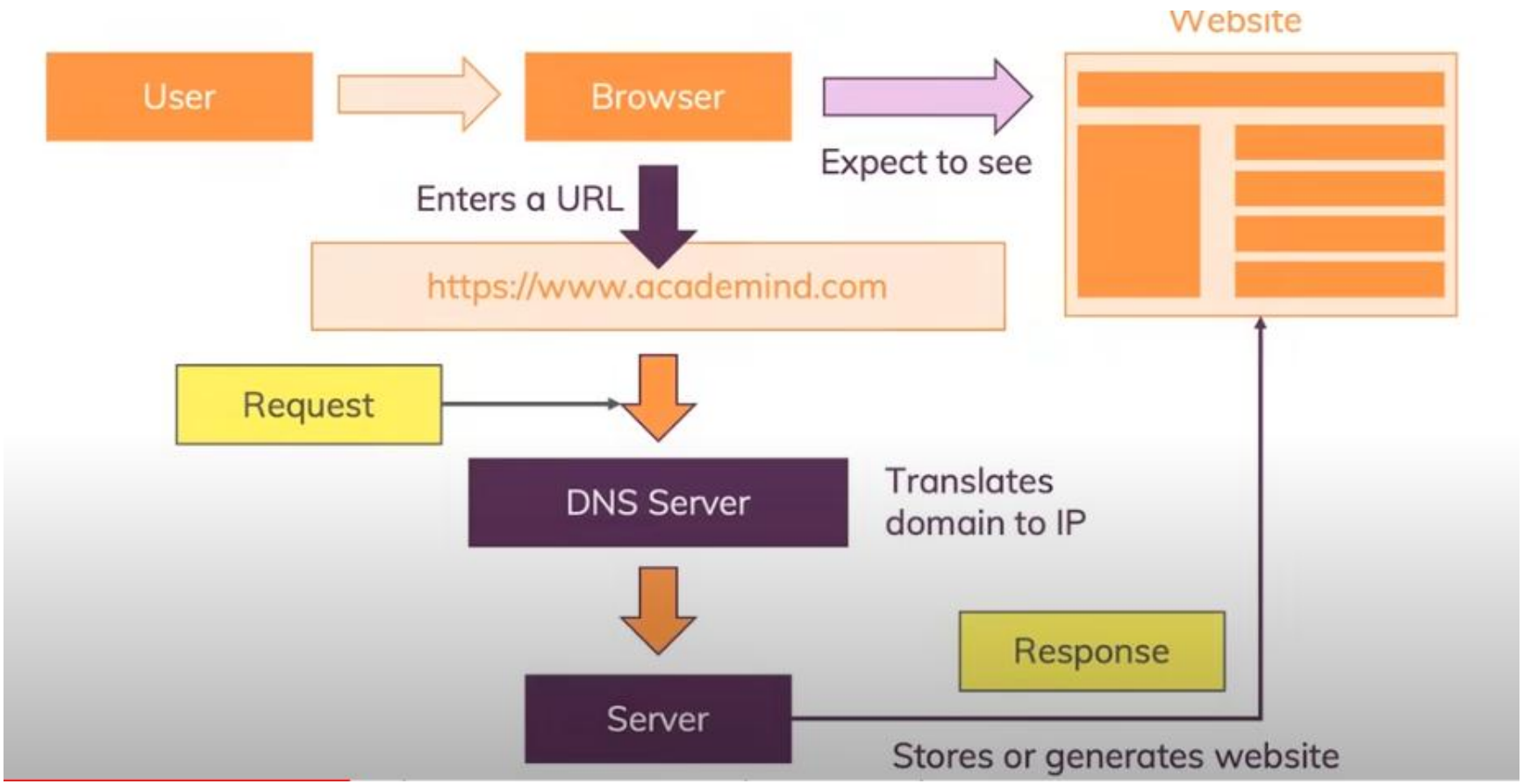
- Choosing your assets

1. Text
2. Theme Color
3. Images
4. Font

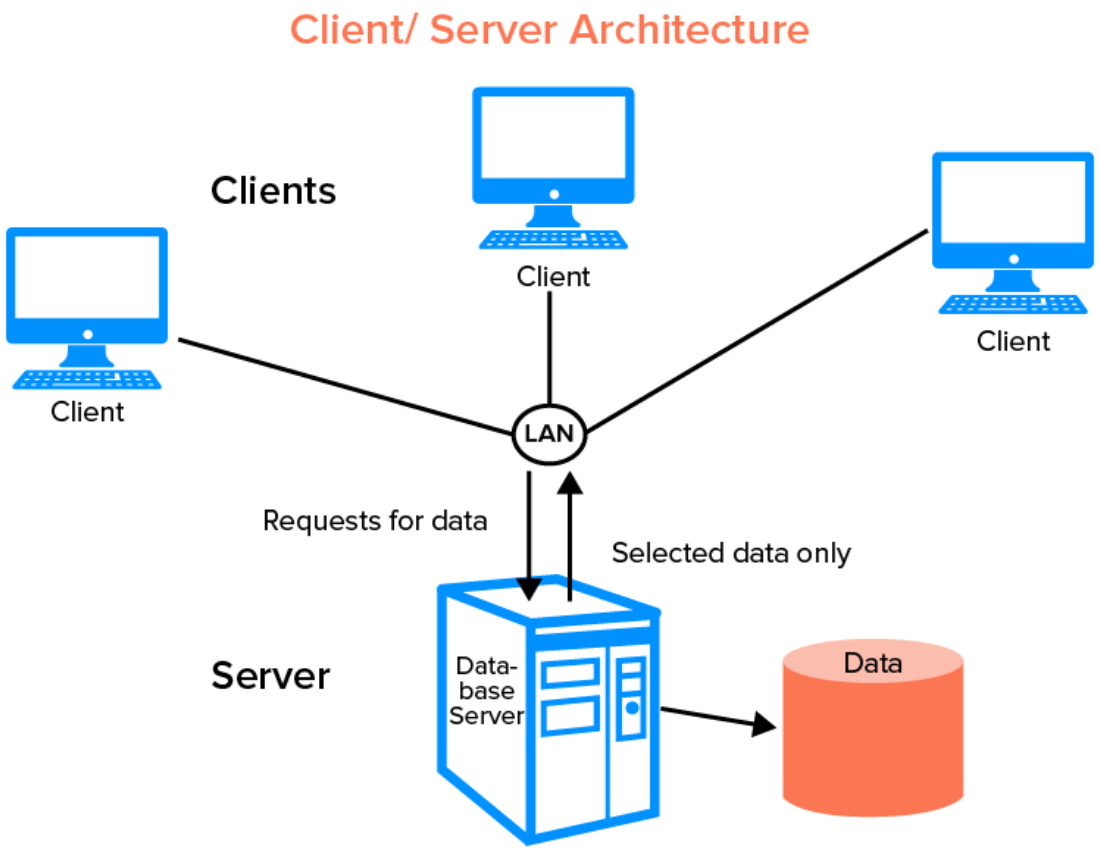
Website Planning

- Where should your website live on your computer?
- What structure should your website have?
- Publish your website

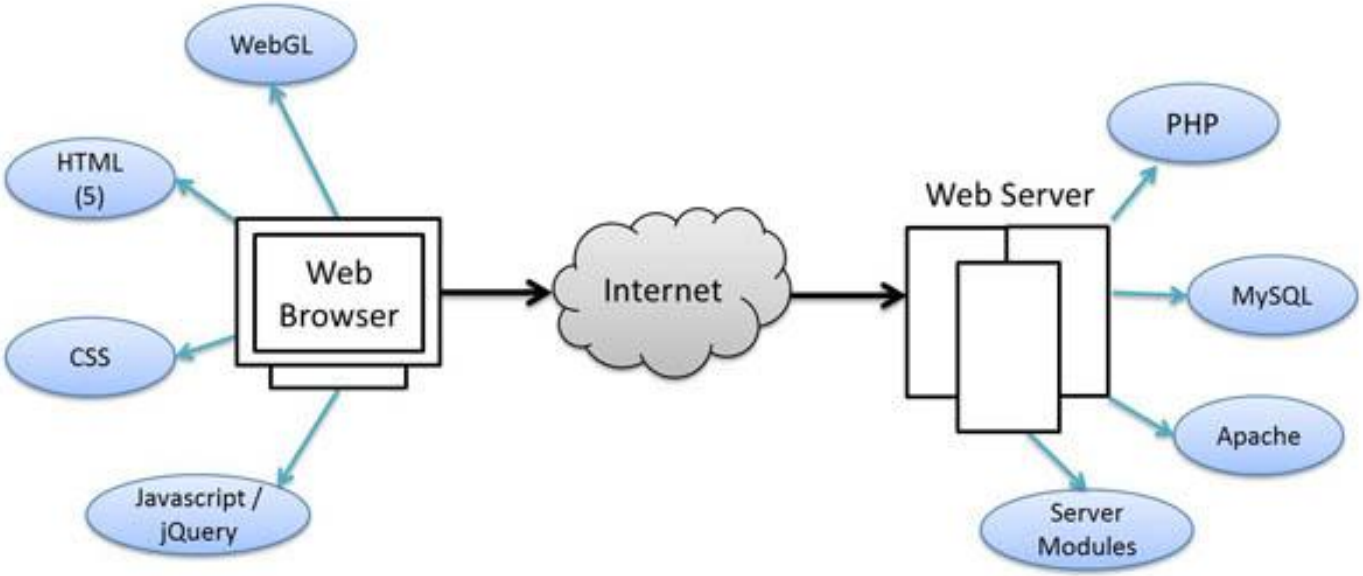
How the Web Works



Client Server Architecture



Technology Involved



Basics of HTML

- Html stands for hyper text markup language, basically used to develop the web pages.
- The hypertext markup language highlights the different views of a given web page in terms of headers, titles, plain text, images, sounds, videos and etc.
- In Html, all these can be performed with the help of tags, where a tag is nothing but a special keyword supported by Html which indicates in between < , > symbols.

Types of tags

In general there are two types of tags.

1. Empty tags / single tags
2. Container tags

Empty Tags

- Empty tags : Empty tag represents the formatting options of text such as the line breaks, horizontal ruler, paragraph and etc..
- These tags has only single tag, mentioned as the start tag.
- Ex: <br>, <hr>

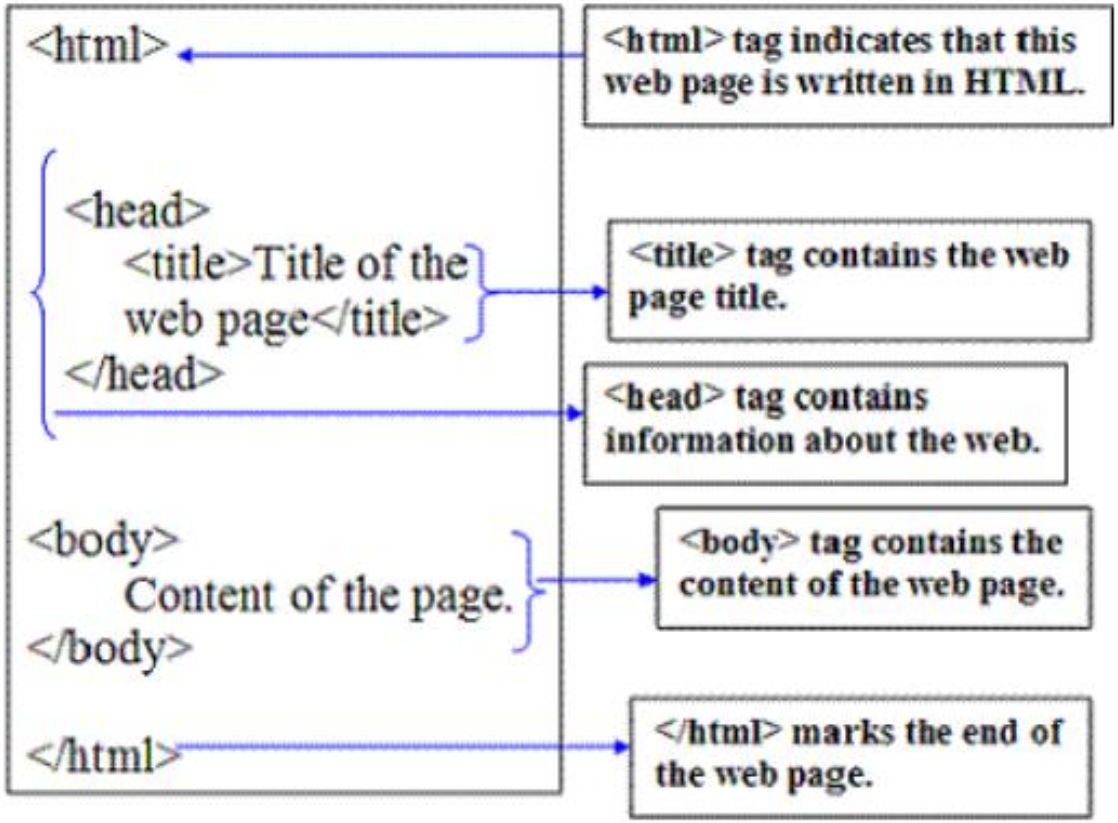
Container Tags

- Container tags : Container tag has a section of text, which specifies the formatting construction for all of the selected text. Container tag has two tags representing the start tag and close tag.
- Ex : <html> …. </html>

  <body> …. </body>

   ….

Structure of HTML Program



Anchor tag & Hyperlinks

- The <a> tag (anchor tag) in HTML is used to create a hyperlink on the webpage.
- This hyperlink is used to link the webpage to other web pages or some section of the same web page.
- It is either used to provide an absolute reference or a relative reference as its “href” value.
- Syntax:

```
<a href=”URL address” target=”frame address”
name=”section name” shape=”circle | rectangle”> …      </a>
```

In the above statement

- href :name of the source file
- target :name of the frame where we divide the webpage into frame

Anchor tag & Hyperlinks

- By default, the linked page will be displayed in the current browser window. To change this, you must specify another target for the link.
- The target attribute specifies where to open the linked document.

- The target attribute can have one of the following values:
- _self - Default. Opens the document in the same window/tab as it was clicked
- _blank - Opens the document in a new window or tab
- _parent - Opens the document in the parent frame
- _top - Opens the document in the full body of the window

- name --- provides the section for the webpage
- Shape --- provides the shape of the area in the image.

Types of hyperlinks

- Absolute hyperlink(External hyperlink)
- Relative hyperlink(Internal hyperlink)

Absolute hyperlink

- An absolute hyperlink uses a complete URL to connect browsers to a webpage or online resource.
- Links that uses a complete URL to point to a resources are called absolute because they provide a complete, stand- alone path to another web resource.
- The following address is an example of a full address in an absolute hyperlink:

  <a href=”http://www.microsoft.com/”>Microsoft</a>

Relative Hyperlink

- A link to a webpage within the same website is referred to as relative hyperlink.
- Create relative links between the resources in the same domain.
- Because both resources are in the same domain you may skip domain information from the URL
- E.g. <a href=”index.html”>Home page</a>

Code Lines

- <!DOCTYPE html>
- <html lang="en">
- <head>
- <meta charset="UTF-8">
- <meta name="viewport" content="width=device-width, initial-scale=1.0">
- <title>Document</title>
- </head>
- <body>
- <h1>Welcome to My Website</h1>
- <p>This is my first webpage. Click the link below to visit <a href="https://www.vit.edu/">Vit College Website</a>. <! Absolute linking>
- <p>For homepage click here <a href="firstprg.html">Homepage</a>. <! Relative Linking>
- </body>
- </html>

Document Body

- The two major blocks of text in HTML document are the **paragraph and heading.**
- **Almost all text and images** in your document will be part of either a heading or a paragraph.

The HTML paragraph tag ( )

1. HTML paragraph tags are used to define the HTML paragraph element.

2. The paragraph element begins with the HTML tag and ends with the HTML tag.

3. The HTML paragraph element should not contain tables and other block elements.

*4. tag having one optional attribute that is **align.***

5. The align attribute specifies the alignment of the text within a paragraph.

- **Syntax: <p [align=”left” | “center” | “right”]> **

  or

- **Syntax: 0**

Attribute Values for Paragraph tag

Value	Description
left	Left-align text
right	Right-align text
center	Center-align text
justify	Stretches the lines so that each line has equal width (like in newspapers and magazines)

HTML Heading tags

- The <h1> to <h6> tags are used to define HTML headings.
- <h1> defines the largest heading and <h6> defines the smallest heading.
- The align attribute specifies the horizontal alignment of a heading.
- **Syntax:**

  <h1 [align=”left” | “center” | “right”]> ..... </h1>

  <h2 [align=”left” | “center” | “right”]>.....</h2>

  <h3 [align=”left” | “center” | “right”]> ..... </h3>

  <h4 [align=”left” | “center” | “right”]>.....</h4>

  <h5 [align=”left” | “center” | “right”]> ..... </h5>

  <h6 [align=”left” | “center” | “right”]>.....</h6>

Horizontal ruler tag - <hr>

- The <hr> tag creates a horizontal line in an HTML page. The hr element can be used to separate content in an HTML page.
- Syntax: <hr [align=”left” | “center” | “right”] [size=”n”] [noshade] [width=”nn %”]>
- **Hr Tag Attributes:**

1. color - Specifies the color of the Horizontal line in the html document.
2. align - Specifies the alignment of a hr element
3. width - Specifies the width of a hr element
4. size - Specifies the height of a hr element, it is given in pixels.
5. noshade - Specifies that a hr element should render in one solid color (noshaded), instead of a shaded color

Code Lines

- <!DOCTYPE html>
- <html lang="en">
- <head>
- <meta charset="UTF-8">
- <meta name="viewport" content="width=device-width, initial-scale=1.0">
- <title>Document</title>
- </head>
- <body>
- <h1>Horizontal Ruler Examples</h1>
- Aligned Left: <hr align="left" width="50%">
- Aligned Center: <hr align="center" width="50%">
- Aligned Right: <hr align="right" width="50%">
- Size Attribute: <hr size="5" width="50%"> No Shade:
- </body>
- </html>

COMMENTING YOUR PAGES

- The comment tag is used to insert a comment in the source code.
- Comments within your source code can be used to explain your code which may help you with editing at a later date. The comment code will be ignored by the browser.
- When you put comments in your HTML code, begin the comment with <!-- end it with -->.

Text formatting tags

- To specify the overall font for your page add the <**font> tag.**
- **The <font>** tag is used to specify the text font size, and optionally the color and font name. The **<font> applies to the text that** follows it. This tag is not supported by some browsers.
- Syntax: <FONT COLOR= "red" FACE="Arial" SIZE="3“>

Code Lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h1>Using the Font Tag</h1>
    <p>This is a paragraph with <font face="Arial" size="4" color="blue">Arial</font> font, size 4, and blue color.</p>
    <p>This is a paragraph with <font face="Verdana" size="5" color="red">Verdana</font> font, size 5, and red color.</p>
    <p>This is a paragraph with <font face="Times New Roman" size="6" color="green">Times New Roman</font> font, size 6, and green color.</p>
</body>
</html>
```

FORMATTING TECHNIQUES USED IN HTML DOCUMENTS

- The text formatting techniques are used to maintain an effective appearance of our web document.

a) **..**

- This tag is used to display the text in **Bold letters.**

b) ** .. **

- This tag is used to display the text in ***Italic letters.***

FORMATTING TECHNIQUES USED IN HTML DOCUMENTS

c) **<u> .. </u>**

- This tag is used to display the text with giving **underline.**

d) ** .. **

- This tag is used to display the text in **new paragraph.**

e) <big> .. </big>

- This tag is used to display the text in **BIG letters.**

f) <small> .. </small>

- This tag is used to display the text in **small letters.**

**g) .. **

- This tag is used to display the text in **STRONG letters.**

FORMATTING TECHNIQUES USED IN HTML DOCUMENTS

h) <strike> .. </strike>

- This tag is used to display the text with strikethrough.

i) <centre> .. </centre>

- This tag is used to display the text in **centre alignment.**

j) <pre> .. </pre>

- This tag is used to display the text in **pre formatted text.**

k) **.. **

- This tag is used to display the text in **emphasis.**

k) <cite>.. </cite>

- This tag is used to display the text in **citation.**

FORMATTING TECHNIQUES USED IN HTML DOCUMENTS

l) ^{..}

- This tag is used to display the text in **super script letters.**

m) _{..}

- This tag is used to display the text in **sub script letters.**

n) <address> .. </address>

- This tag is used to display the text as **address line by line.**

o) <blink> .. </blink>

- This tag is used to display the text in **blinking letters.**

p) <marquee> .. </marquee>

- This tag is used to display the text as **scrolling letters.**

LISTS in HTML

- The list refers to arrange the elements in a group, or arrange in a specific format.
- The html supports **FOUR types of lists.**

1. unordered lists

2. order lists

3. miscellaneous lists or Nested lists

4. definition lists

- The most common HTML lists are **ordered and unordered lists**

Unordered Lists

- If we intend to display certain text on the webpage with each independent sentence being initiated with out any sequence, then we represent them using unordered list.
- An unordered list starts with the <ul> tag. Each list item starts with the <li> tag, and ends with </li>. The
- Unordered list ends with </ul> . The list items are marked with bullets (typically small black circles).
- **Syntax : <ul type=”disc” | “square” | ”circle” >……..</ul>**

Ordered Lists

- If we intend to display certain text on the webpage with each independent sentence being initiated with in an order or sequence, then we represent them using ordered list.
- An ordered list starts with the <ol> tag. Each list item starts with the <li> tag, and ends with </li>. The unordered list ends with </ol> .
- The list items are marked with numbers.
- **Syntax :**

```
<ol>
<li> item1 </li>
<li> item2 </li>
<li> item3 </li>
</ol>
```

Code Lines

```
!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Document</title>
</head>
<body>
   <h1> science courses</h1>
<ul>
<li> B.Sc CPM </li>
<li> B.Sc MEC </li>
<li> B.Sc MPC</li>
<li> B.Sc CBZ</li>
</ul>
<h1> science courses</h1>
<ol>
<li> B.Sc CPM </li>
<li> B.Sc MEC </li>
<li> B.Sc MPC</li>
<li> B.Sc CBZ</li>
</ol>
</body>
</html>
```

Miscellaneous Lists / Nested Lists

- If we intend to display certain text on the webpage with each independent sentence being initiated with order or sequence, and also contains the unordered, then we represent them using Nested list.

Nested Lists

- <ol>
- <li>Coffee</li>
- <ul>
- <li> coffee powder </li>
- <li> milk </li>
- <li> sugar </li>
- </ul>
- <li>Tea </li>
- <ul>
- <li> tea powder </li>
- <li> milk </li>
- <li> sugar </li>
- </ul>
- </ol>

Definition List (DL)

- **<dl>….. </dl>**
- This tag is used for the third category i.e., definition list, where numbers or bullet is not used in front of the list item, instead it uses definition for the items.

COLORS

- A web page can be made up colourful by using some colour attributes in the body tag.
- **Back ground design**
- **Back ground colour**
- **Text colour**

Back ground design

- The back ground design of the page can be designed with any available picture, with its formats like bmp, gif, jpg, jpeg, etc. It can be represented in body tag.
- Syntax : <body src= “ path of the picture location ”>

  Or

  <body src= “ pic name.format”>.

Background colour

- In general, any colour is the combination of three basic colours. They are rgb (red, green, blue).
- In html each of the above colour has a degree from 00 to FF in hexadecimal form that permits 256 colours.
- We can assume a six digit format, using with #, like #rgb
- #000000 to #FFFFFF
- If we assign the #00FF00, i.e. red=00, green=FF, blue=00 then it will display green. In this, the base colors.
- So, we can create 256x256x256 = 1,67,77,216 colors to use.

- The general form is <body bgcolor = “colour name”>
- Ex: <body bgcolor = “red”> OR

  <body bgcolor = “#FF0000”>

Text colour

- The colour of the text in the web page can be selected by assigning a color string to the text/font attribute.
- We can assign the text/font attribute in the <body>.
- The general form is <body text/font = “colour name”>
- Ex: <body text/font = “red”>

  <body text/font = “#FF0000”>

IMAGES

- Images are used to increase the visibility and shows the effectiveness of our webpage.
- Including the graphics in a page will make the web more attractive.
- Html provides a unique way to include any type of graphic file into the document, but many browsers support only gif(graphic interchange format), jpg (joint photographic group) and jpeg (joint photographic expert group) formats.
- The <img> tag creates a holding space for the referenced image.

IMAGES

- **Syntax :**
  <img src=”URL”|”name”
  [alt=”string”] [align=”top”|”center”|”bottom”] [usemap=”URL]>
  ….
  </img>

  The <img> tag has mainly two required attributes: **src and alt.**
- **src: src stands for source. It refers to the filename of an image. It specifies the URL(uniform resource**
- locator), where the image is actually stored in the web server or in the computer system.
- **alt : alt stands for alternate. Some browsers can not display images or sometimes it takes long time to**
- display the image. In such cases, the ALT attribute is used to display an alternate message.

IMAGES

- **Optional Attributes**
- **1. align : It is used to align or arrange the image on the web page. These values can be**
- left/right/centre/top/bottom.
- **2. border : It is used to set the border to an image. The default value is zero.**
- **3. height / width : height and width of the image can be specified using this attribute. These values**
- can be used in pixels %.
- **4. hspace / vspace : using this attribute, the white spaces or gaps existing between the text and the**
- image can be managed.
- **5. name : usually, this attribute used to specify the name of the image**

Example of **IMAGES**

- **Ex : <img src="angry.gif" alt="Angry face" align= centre width="32" height="32" border=5>**

Table handling in HTML

- In HTML, Tables are defined with the <table> tag.
- Table is divided into rows; each row divided into data cells.
- The data cells contains text, images, lists, paragraphs, forms, horizontal rules, tables etc.

Table handling in HTML

- The most important layout aspects that can be done with tables are:
- Divide the page into different sections.
- Creating menus.
- Adding interactive form fields.
- Easy alignment of image that have been cut into smaller pieces.
- A simple way to allow text to be written in two or more columns next to each other.

Syntax for craeting table

- **Syntax:**
- <table [align=”Left” | “Center” | “Right” | “Justify”] [bgcolor=”color”] [border=”n”]
- [Cellpadding=”n”] [cellspacing=”n”] [width=”n%”] [height=”n%”] >

Attributes of table tag

Attribute	Value	Description
Border	Pixels	Specifies the border width. A value of "0" means no border
Bordercolor	Color name	Specifies the color of the table's border.
Align	Right/left/center	Specifies the alignment of a table
Bgcolor	Color name	Specifies the background color of the table.
Background	URL	Specifies the background image for the table.
Height	Pixels or %	Specifies the height of the table
Width	Pixels or %	Specifies the width of the table
Cellspacing	Pixels or %	Specifies the space between cells.
Cellpadding	Pixels or %	Specifies the space between the cell borders and their contents

- **Table Row <tr>: Each row of the table is created . It has the following form**
- **Syntax : <tr align=”center”|”left”|”right”][valign=”top”|”center”|””bottom]> …. </tr>**

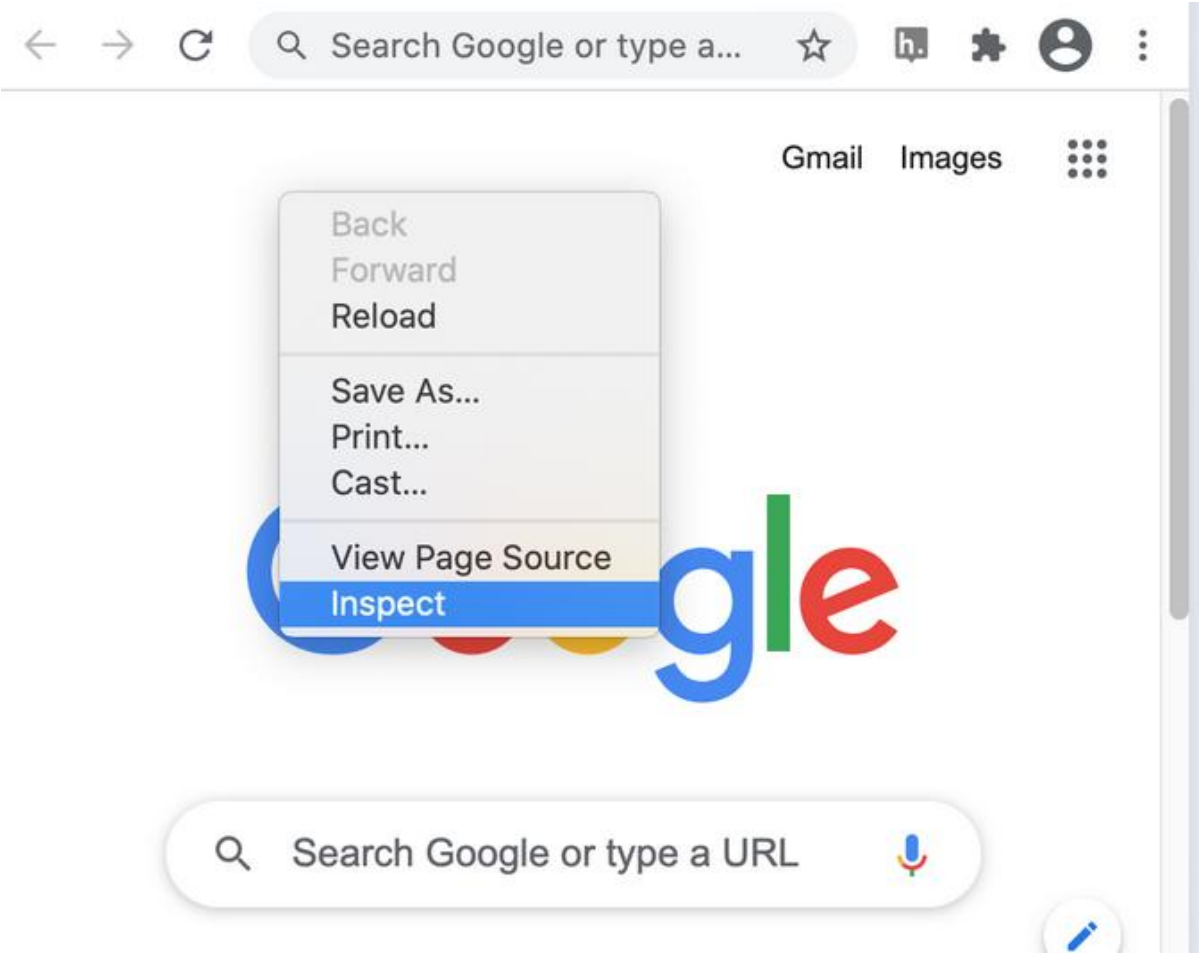
- **Table Data <td>: used to represent table cell and it data.**
- **Syntax :** <td align=”center”|”left”|”right”][valign=”top”|”center”|””bottom][nowrap][colspan=”n”][rowspan=”n”]>
- ….
- </td>

- **Table Heading <th>: Used to create headings for columns in a table.**
- **Syntax :**<th align=”center”|”left”|”right”][valign=”top”|”center”|””bottom][nowrap][colspan=”n”][rowspan=”n”]>
- …. </th>

The attributes of <tr>, <td>, <th> are describe below

Attribute	Description
Align	Used to align the contents horizontally
Valign	Used to align the content vertically
Nowrap	If Nowrap is set the content will not be automatically wrapped as the table is formatted for the screen
Colspan	This controls how many columns a text in the table should span
Rowspan	This defines the number of rows the text in the table should span

Java Script


Search Google or type a...
Gmail
Images
Back
Forward
Reload
Save As...
Print...
Cast...
View Page Source
Inspect
Search Google or type a URL

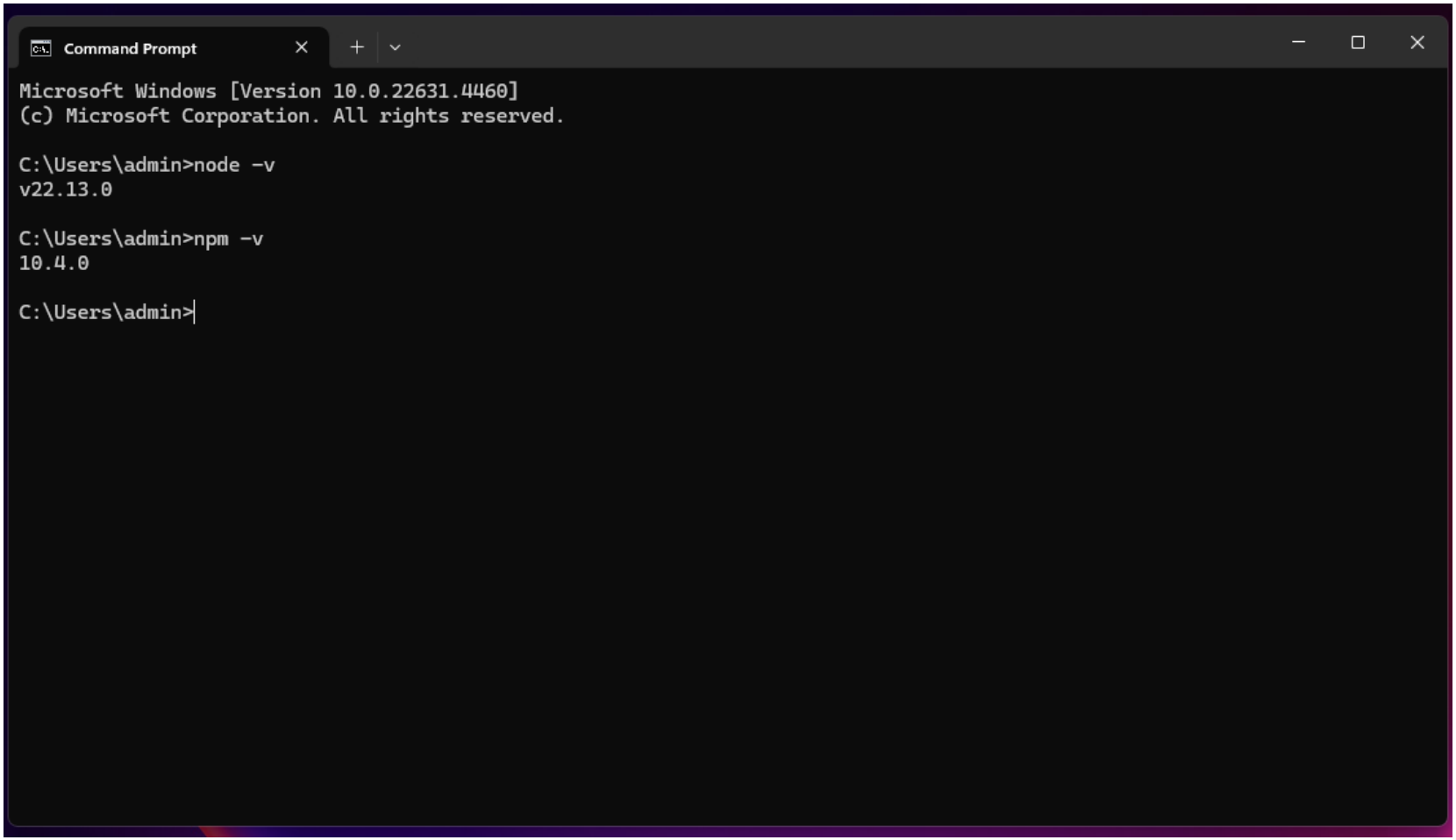
JavaScript

- JavaScript was initially created to make web pages interactive, that's why JavaScript and HTML go hand in hand.

Node js

- Node.js is a powerful, open-source, and cross-platform JavaScript runtime environment built on Chrome's V8 engine.
- It allows you to run JavaScript code outside the browser, making it ideal for building scalable server-side and networking applications.
- JavaScript was earlier mainly used for frontend development. With Node JS (Introduced in 2009), JavaScript became a backend language as well.
- Non-blocking, event-driven architecture for high performance.
- Supports the creation of REST APIs, real-time applications, and microservices.
- Comes with a rich library of modules through npm (Node Package Manager).

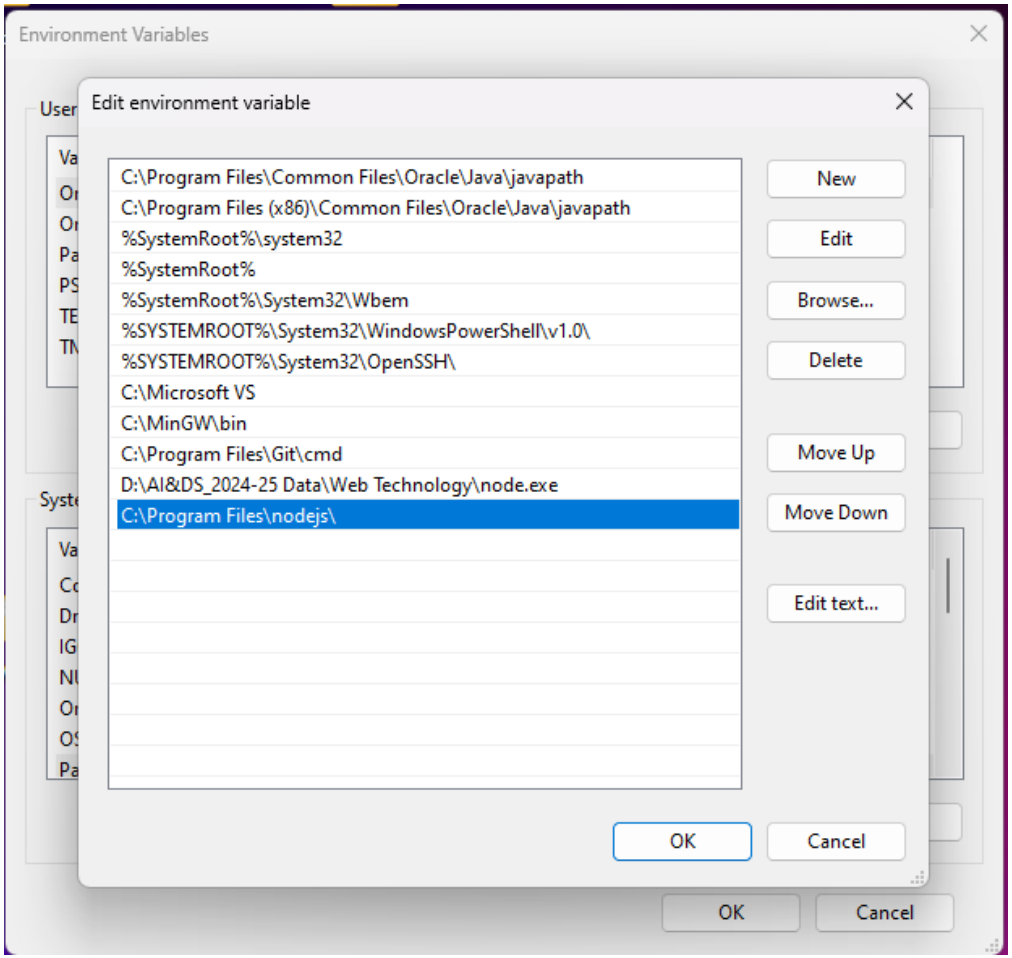
Check installation of Node js on cmd



Check the path

- Find environment variables, ---→ select system variables---→ select path

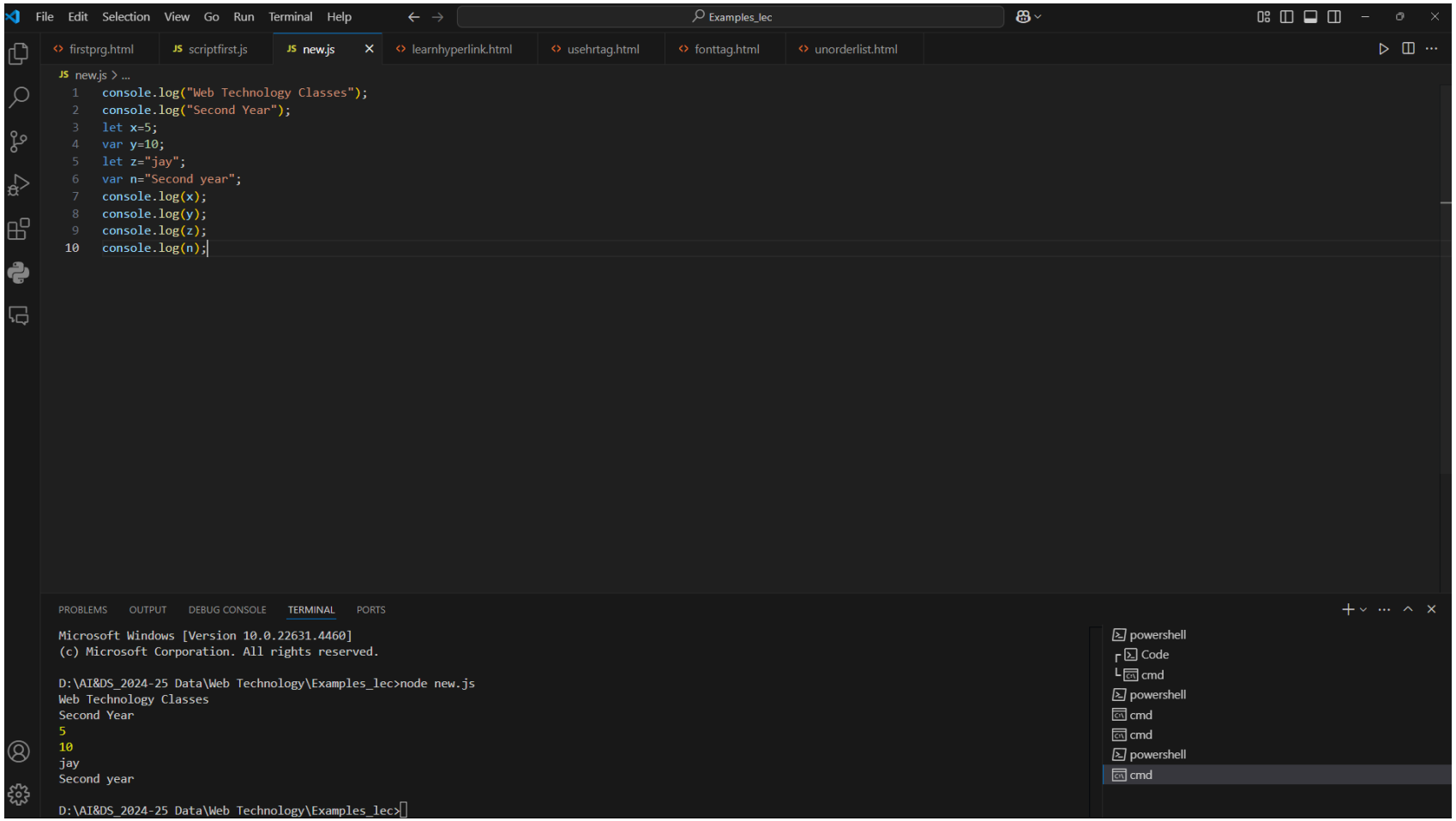
Check the path



Visual Studio (**create a file with below code** > **write JS code** > **save it with .js extension**.)

- console.log(“ Web Technology Classes”);
- console.log(“Second Year”);

Code to display



main.js

```
1  let x=5;
2  var y=10;
3  let z="Vyom";
4  var n="Second Year";
5  console.log(x+' '+y);
6  console.log (z + ' ' + n);
7
8
9
```

Arithmetic operators

main.js

```
1  let x = 5;
2  let y = 3;
3  console.log('x + y = ', x + y);
4  console.log('x - y = ', x - y);
5  console.log('x * y = ', x * y);
6  console.log('x / y = ', x / y);
7  console.log('x % y = ', x % y);
8  console.log('++x = ', ++x);
9  console.log('x++ = ', x++);
10 console.log('x = ', x);
11 console.log('--x = ', --x);
12 console.log('x-- = ', x--);
13 console.log('x = ', x);
14 console.log('x ** y =', x ** y);
```

Output

```
node /tmp/GG3oxZb9gD.js
x + y =  8
x - y =  2
x * y =  15
x / y =  1.6666666666666667
x % y =  2
++x =  6
x++ =  6
x =  7
--x =  6
x-- =  6
x =  5
x ** y = 125
```

Comparison operators

main.js

```
// equal operator
console.log(2 == 2);
console.log(2 == '2');

// not equal operator
console.log(3 != 2);
console.log('hello' != 'Hello');

// strict equal operator
console.log(2 === 2);
console.log(2 === '2');

// strict not equal operator
console.log(2 !== '2');
console.log(2 !== 2);
```

Output

```
node /tmp/7x3voSYYen.js
true
true
true
true
true
false
true
false
```

Logical operators

main.js

```
1  // logical AND
2  console.log(true && true);
3  console.log(true && false);
4
5  // logical OR
6  console.log(true || false);
7
8  // logical NOT
9  console.log(!true);
```

Output

```
node /tmp/Ol
true
false
true
false
```

String concatenate

if-else

```
if (condition) {
  // Code to be executed if the condition is true
} else {
  // Code to be executed if the condition is false
}
```

main.js

```
1  let number = 8;
2
3  if (number % 2 === 0) {
4    console.log(number + " is an even number.");
5  } else {
6    console.log(number + " is an odd number.");
7  }
8
```

Output

```
node /tmp/axSRQLAEKB.js
8 is an even number.
```

main.js

```
1  let score = 75;
2
3  if (score >= 90) {
4    console.log("A");
5  }
6  else if (score >= 80) {
7    console.log("B");
8  }
9  else if (score >= 70) {
10   console.log("C");
11 }
12 else {
13   console.log("F");
14 }
```

Output

```
node /tmp/OlB3pKkmLv.js
C
```

```
let num1 = 15;
let num2 = 24;
let num3 = 10;

if (num1 >= num2) {
  if (num1 >= num3) {
    console.log(num1 + " is the maximum number.");
  } else {
    console.log(num3 + " is the maximum number.");
  }
} else {
  if (num2 >= num3) {
    console.log(num2 + " is the maximum number.");
  } else {
    console.log(num3 + " is the maximum number.");
  }
}
```

Leap Year

For Loop

```
for (initialExpression; condition; updateExpression) {
    // for loop body
}
```

main.js

```
1  const n = 5;
2  for (let i = 1; i <= n; i++) {
3      console.log(i*n);
4  }
5
6
7
8
9
10
```

Output

```
node /tmp/1Fhup3HuAY.js
5
10
15
20
25
```

```
for (let i = 1; i <= 10; console.log(i++));
```

```
let i;
for (i = 0; i++ <= 10; console.log(i));
```

while Loop

```
while (condition) {
    // loop body
}
```

main.js

```
1  let i = 1, n = 5;
2  while (i <= n) {
3      console.log(i);
4      i += 1;
5  }
```

Output

```
node /tmp/7MGRGx4ruy.js
1
2
3
4
5
```

Do while Loop

```
do {
    // loop body
} while (condition);
```

main.js

```
1  let i = 1;
2  const n = 5;
3  do {
4      console.log(i);
5      i++;
6  } while(i <= n);
```

Output

```
node /tmp/r4dWZNgGFN.js
1
2
3
4
5
```

main.js

```
1  let n=5;
2  for (let i = 1; i <= n; i++) {
3      if (i == 3) {
4          break;
5      }
6      console.log(i);
7  }
```

Output

```
node /tmp/azl
1
2
```

```
let i = 10;
for(        ;        ;        )
{
    if (i==0)
    break;
    console.log(5*i);
    i--;
}
```

```
let i ;
for( i=1 ; i<=10 ;
i++) ;
{
    console.log(i);
}
```

```
let i =1;
while(i<=10);
{

console.log(i++);
}
```

The continue Statement

- The continue statement skips the current iteration of the loop and continues with the next iteration.
- When a continue statement is encountered inside a loop, control jumps to the beginning of the loop for next iteration, skipping the execution of statements inside the body of loop for the current iteration.

main.js

```
1  const n = 17;
2  let i=1;
3  for(let i=1;i<n;i++){
4      if(i%3==0)
5      {
6          continue;
7      }
8      console.log(i);
9  }
```

Output

```
node /tmp/X7dCxjaDIc
1
2
4
5
7
8
10
11
13
14
16
```

```
1   const n = 17;
2   let i=1;
3   while(i<n) {
4       if(i%3==0)
5       {
6           continue;
7       }
8       console.log(i);
9       i++;
10  }
```

```
node /tmp/
1
2
```

main.js

```
1   const n = 17;
2   let i=1;
3   while(i<n) {
4       if(i%3==0)
5       {
6           i++;
7           continue;
8       }
9       console.log(i);
10      i++;
11  }
```

Output

```
node /tmp/0iR05JSX1w.js
1
2
4
5
7
8
10
11
13
14
16
```

Switch

```
switch(variable/expression) {
    case value1:
        // body of case 1
        break;
    case value2:
        // body of case 2
        break;
    case valueN:
        // body of case N
        break;
    default:
        // body of default
}
```

main.js

```
// program using switch statement
let a = 2;
switch (a) {
    case 1:
        a = 'one';
        break;
    case 2:
        a = 'two';
        break;
    default:
        a = 'not found';
        break;
}
console.log('The value is='+ a);
```

Output

```
node /tmp/KVSPPR2DUx.js
The value is=two
```

main.js

```js
// program using switch statement
let a = 2;
switch (a) {
    case 1:
        console.log('Case 1');
    case 2:
        console.log('Case 2');
    case 3:
        console.log('Case 3');
    default:
        console.log('Default Case');
        break;
}
```

Output

```
node /tmp/KVSPPR2DUx.js
Case 2
Case 3
Default Case
```

main.js

```js
// multiple case switch program
let fruit = 'mango';
switch(fruit) {
    case 'apple':
    case 'mango':
    case 'pineapple':
        console.log(`${fruit} is a fruit.`);
        break;
    default:
        console.log(`${fruit} is not a fruit.`);
        break;
}
```

Output

```
node /tmp/kF5pR2Cpmq.js
mango is a fruit.
```

```
a = 2;
switch (a)
{
console.log('I am in switch’);
case 1:        console.log('Case 1');
break;
case 2:
console.log('Case 2');        break;
default:
console.log('Default Case');
break;
}
```

main.js

```
1  let numbers = [10, 30, 40, 60, 80]
2
3  console.log(numbers[0]);
4
5  console.log(numbers[2]);
6
7
```

Output

```
node /tmp/dbVLRKk7L2.js
10
40
```

main.js

```
1  let numbers = [10, 30, 40, 60, 80]
2
3  console.log(numbers[0]);
4
5  console.log(numbers[2]);
6
7  console.log(numbers.length);
```

Output

```
node /tmp/c
10
40
5
```

main.js

```
1  let numbers = [10, 30, 40, 60, 80]
2
3  numbers.push(1);
4
5  console.log(numbers);
6
7
```

Output

```
node /tmp/dbVLRKk7L2.js
[ 10, 30, 40, 60, 80, 1 ]
```

main.js

```
1  let numbers = [10, 30, 40, 60, 80]
2
3  numbers.unshift(1);
4
5  console.log(numbers);
6
7
```

Output

```
node /tmp/dbVLRKk7L2.js
[ 1, 10, 30, 40, 60, 80 ]
```

main.js

```
1  let routine = ['eat', 'sleep'];
2
3  routine.push('exercise');
4  routine.unshift('work');
5
6  console.log(routine);
```

Output

```
node /tmp/plAk6ROgQ8.js
[ 'work', 'eat', 'sleep', 'exercise' ]
```

main.js

```
1  var arr = [12, 11, 13, 5, 6];
2  var len = arr.length;
3    for (var i = 1; i < len; i++) {
4      var key = arr[i];
5      var j = i - 1;
6      while (j >= 0 && arr[j] > key) {
7        arr[j + 1] = arr[j];
8        j--;
9      }
10
11     arr[j + 1] = key;
12   }
13 console.log("Sorted array:", arr);
14
```

Output

```
node /tmp/dbVLRKk7L2.js
Sorted array: [ 5, 6, 11, 12, 13 ]
```

Insertion Sort

```
var arrayToSort = [12, 11, 13, 5, 6];
var len = arr.length;
for (var i = 1; i < len; i++) {
    var key = arr[i];
   var j;
    for (j = i - 1; j >= 0 && arr[j] > key; j--) {
      arr[j + 1] = arr[j];
    }
arr[j + 1] = key;
  }
console.log("Sorted array:", arr);
```

Functions

Introduction

- **Functions** are one of the fundamental building blocks in JavaScript.
- A function is a reusable set of statements to perform a task or calculate a value.
- Functions can be passed one or more values and can return a value at the end of their execution.
- In order to use a function, it must be defined somewhere in the scope in which it will be utilized.

Function Declaration

Function declarations are built from:

- The function keyword.
- The function name.
- An optional list of parameters separated by commas enclosed by a set of parentheses ().
- A function body enclosed in a set of curly braces {}.

Example

```
function sum(number1, number2) {
  console.log(number1 + number2);
}
```

Calling Functions

- Functions can be called, or executed, elsewhere in code using parentheses following the function name.
- When a function is called, the code inside its function body runs.
- Arguments are values passed into a function when it is called.

Example

```
// Defining the function
function sum(num1, num2) {
  return num1 + num2;
}

// Calling the function
console.log(sum(2, 4));
// Output 6
```

Return Keyword

- Functions return (pass back) values using the return keyword.
- return ends function execution and returns

ere it

```
// With return
function sum(num1, num2) {
  return num1 + num2;
}

// Without return, so the function doesn't output the sum
function sum(num1, num2) {
  num1 + num2;
}
```

Arrow Functions

- The syntax for an arrow function expression does not require the function keyword and uses a fat arrow => to separate the parameter(s) from the body.
- There are several variations of arrow functions:

1. Arrow functions with a single parameter do not require () around the parameter list.
2. Arrow functions with a single expression can use the concise function body which returns

Arrow function with no arguments:

```
const printHello = () => {
  console.log('hello');
};

printHello();
// Output: hello
```

Arrow function with a single argument:

```
const checkWeight = (weight) => {
  console.log(`Baggage weight : ${weight} kilograms.`);
};

checkWeight(25);
// Output: Baggage weight : 25 kilograms.
```

Arrow function with two arguments:

```
const sum = (firstParam, secondParam) => {
  return firstParam + secondParam;
};

console.log(sum(2, 5));
// Output: 7
```

Concise arrow function:

```
const multiply = (a, b) => a * b;

console.log(multiply(2, 30));
// Output: 60
```

Embedding functions in html

```html
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>
<p>Using a function as a variable:</p>

<p id="demo"></p>

<script>
let text = "The temperature is " + toCelsius(77) + " Celsius.";
document.getElementById("demo").innerHTML = text;

function toCelsius(fahrenheit) {
  return (5/9) * (fahrenheit-32);
}
</script>

</body>
</html>
```

JavaScript Functions

Using a function as a variable:

The temperature is 25 Celsius.

Embedding Arrow functions in html

```html
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>
<h2>The Arrow Function</h2>

<p>This example shows an Arrow Function with a parameter.</p>

<p id="demo"></p>

<script>
let hello = "";

hello = (val) => "Hello " + val;

document.getElementById("demo").innerHTML = hello("Universe!");
</script>

</body>
</html>
```

JavaScript Functions

The Arrow Function

This example shows an Arrow Function with a parameter.

Hello Universe!

Arrays

- An array is a special variable, which can hold more than one value.
- An array can hold many values under a single name, and you can access the values by referring to an index number.
- Arrays can be created by using square brackets [], with individual elements separated by comma.
- A declaration can span multiple lines

Creating an Array

```html
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>

<p id="demo"></p>

<script>
const cars = ["Saab", "Volvo", "BMW"];
document.getElementById("demo").innerHTML = cars;
</script>

</body>
</html>
```

JavaScript Arrays

Saab,Volvo,BMW

Using the JavaScript Keyword new

```html
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>

<p id="demo"></p>

<script>
const cars = new Array("Saab", "Volvo", "BMW");
document.getElementById("demo").innerHTML = cars;
</script>

</body>
</html>
```

JavaScript Arrays

Saab,Volvo,BMW

Changing an Array Element

```html
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>Bracket Indexing</h2>

<p>JavaScript array elements are accessed using numeric indexes (starting
from 0).</p>

<p id="demo"></p>

<script>
const cars = ["Saab", "Volvo", "BMW"];
cars[0] = "Opel";
document.getElementById("demo").innerHTML = cars;
</script>

</body>
</html>
```

JavaScript Arrays

Bracket Indexing

JavaScript array elements are accessed using numeric indexes (starting from 0).

Opel,Volvo,BMW

Converting an Array to a String

- The JavaScript method toString() converts an array to a string of (comma separated) array

```html
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Arrays</h1>
<h2>The toString() Method</h2>

<p>The toString() method returns an array as a comma separated string:
</p>

<p id="demo"></p>

<script>
const fruits = ["Banana", "Orange", "Apple", "Mango"];
document.getElementById("demo").innerHTML = fruits.toString();
</script>

</body>
</html>
```

JavaScript Arrays

The toString() Method

The toString() method returns an array as a comma separated string:

Banana,Orange,Apple,Mango

Array Properties and Methods

The length Property

The length property of an array returns the length of an array (the number of array elements).

Accessing the First Array Element

```
const fruits =
["Banana", "Orange", "Apple", "Mango"];
let fruit = fruits[0];
```

Accessing the Last Array Element

```
const fruits =
```

Adding Array Elements

- The easiest way to add a new element to an array is using the push() method.

```
const fruits = ["Banana", "Orange", "Apple"];
fruits.push("Lemon");  // Adds a new element (Lemon) to fruits
```

Objects

- A JavaScript **object** is a non-primitive data type that can assume the properties and behaviors required for a given programming need.

Property Key	Property Value
firstName	"Elizabeth"
lastName	"Harmon"
age	22
eyeColor	"Hazel"

Example of Object

```
const person = {
  firstName: 'Elizabeth',
  lastName: 'Harmon',
  age: 22,
  eyeColor: 'Hazel',
};
```

Accessing Object Properties

- Object properties can be accessed using the dot notation:

  objectName.propertyName

Example: person.lastName;

Object Methods

- Methods are stored in properties as function definitions.
- The greeting key holds a function value that returns a template literal using backticks.
- When person.greeting value is invoked, the corresponding function will run.

```
const person = {
  firstName: 'Elizabeth',
  lastName: 'Harmon',
  age: 22,
  eyeColor: 'Hazel',
  greeting: function () {
    return `Hi, I am ${this.firstName} ${this.lastName}.`;
  },
};

console.log(person.greeting());
```

Object Methods

```
const person = {
  firstName: 'Elizabeth',
  lastName: 'Harmon',
  age: 22,
  eyeColor: 'Hazel',
  greeting: function () {
    return `Hi, I am ${this.firstName} ${this.lastName}.`;
  },
};

console.log(person.greeting());
```

A constructor function initializes a set of variables at the creation of a class.

```
class Person {
  constructor(firstName, lastName) {
    this.firstName = firstName;
    this.lastName = lastName;
  }
  greeting() {
    return `Hi, I am ${this.firstName} ${this.lastName}.`;
  }
}
```

```
const person = new Person('Elizabeth', 'Harmon');
console.log(person);
/*
```

JavaScript **this** Keyword

- In JavaScript, the this keyword refers to an **object**.
- The this keyword refers to **different objects** depending on how it is used:

1. In an object method, this refers to the **object**.
2. Alone, this refers to the **global object**.
3. In a function, this refers to the **global object**.
4. In a function, in strict mode, this is undefined.
5. In an event, this refers to the **element** that received the event. Methods

this in a Method

```
<html>
<body>

<h1>The JavaScript <i>this</i> Keyword</h1>
<p>In this example, <b>this</b> refers to the <b>person</b> object.</p>
<p>Because <b>fullName</b> is a method of the person object.</p>

<p id="demo"></p>

<script>
// Create an object:
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  fullName : function() {
    return this.firstName + " " + this.lastName;
  }
};

// Display data from the object:
document.getElementById("demo").innerHTML = person.fullName();
</script>
```

this Alone

- When used alone, this refers to the **global object**.
- Because this is running in the global scope.

this Alone

```html
<!DOCTYPE html>
<html>
<body>

<h1>The JavaScript <i>this</i> Keyword</h1>

<p>In this example, <b>this</b> refers to the window object:</p>

<p id="demo"></p>

<script>
let x = this;
document.getElementById("demo").innerHTML = x;
</script>

</body>
</html>
```

The JavaScript *this* Keyword

In this example, **this** refers to the window object:

[object Window]

this in a Function (Default)

```html
<!DOCTYPE html>
<html>
<body>

<h1>The JavaScript <i>this</i> Keyword</h1>

<p>In this example, <b>this</b> refers to the the window object:</p>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = myFunction();

function myFunction() {
  return this;
}
</script>

</body>
</html>
```

The JavaScript *this* Keyword

In this example, **this** refers to the the window object:

[object Window]

this in a Function (Strict)

```html
<!DOCTYPE html>
<html>
<body>

<h1>The JavaScript <i>this</i> Keyword</h1>

<p>In a function, by default, <b>this</b> refers to the global object.</p>

<p>Strict mode does not allow default binding, so <b>this</b> is:</p>
<p id="demo"></p>

<script>
"use strict";
document.getElementById("demo").innerHTML = myFunction();

function myFunction() {
  return this;
}
</script>

</body>
</html>
```

The JavaScript *this* Keyword

In a function, by default, **this** refers to the global object.

Strict mode does not allow default binding, so **this** is:

undefined

Objects

- **.assign()**
- The **Object.assign()** method is used to modify a “destination” object with the contents of one or more objects passed to the method. If there are duplicate keys between the objects the values in the destination object will be overridden in the assignment.
- Syntax

  Object.assign(destinationObj, sourceObj) // Or Object.assign(destinationObj, sourceObj, ...

Objects

- **.create()**
- In JavaScript, the .create() method creates a new object with the specified prototype object and optional additional properties. It essentially allows developers to create an object that inherits properties from another object.

  Syntax

  Object.create(prototype[, propertiesObject])
- prototype: The object that serves as the blueprint for the newly created object.
- propertiesObject (Optional): The object whose properties are added to the newly created object

DOM

- When a web page is loaded, the browser creates a **D**ocument **O**bject **M**odel of the page.
- **The HTML DOM (Document Object Model)** is a programming interface that represents the structure of a web page in a way that programming languages like JavaScript can understand and manipulate.
- A tree of objects where each part of your HTML document (elements, attributes,

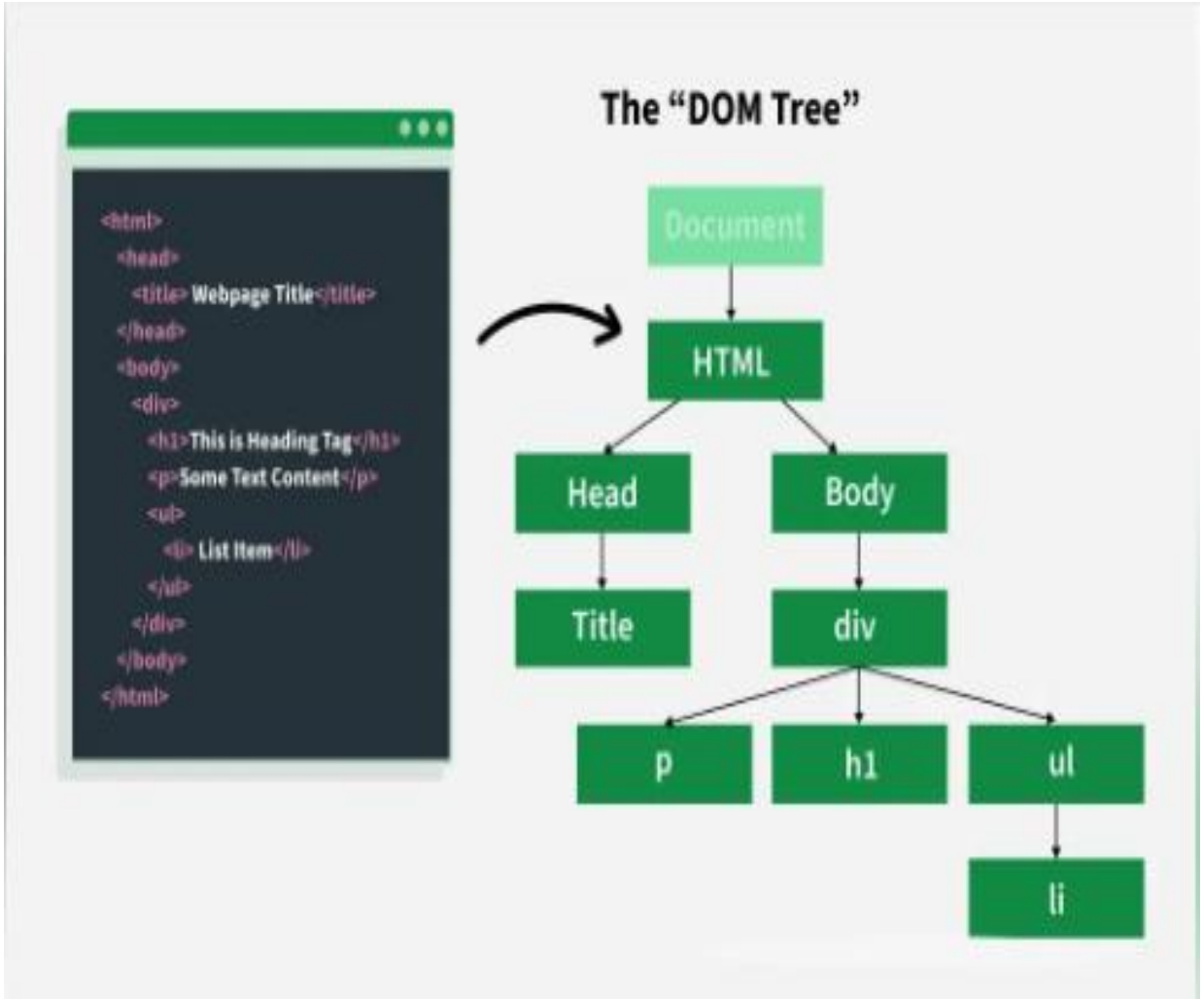
What is the HTML DOM?

The HTML DOM is a standard for how to get, change, add, or delete HTML elements.

- The HTML elements as **objects**
- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements

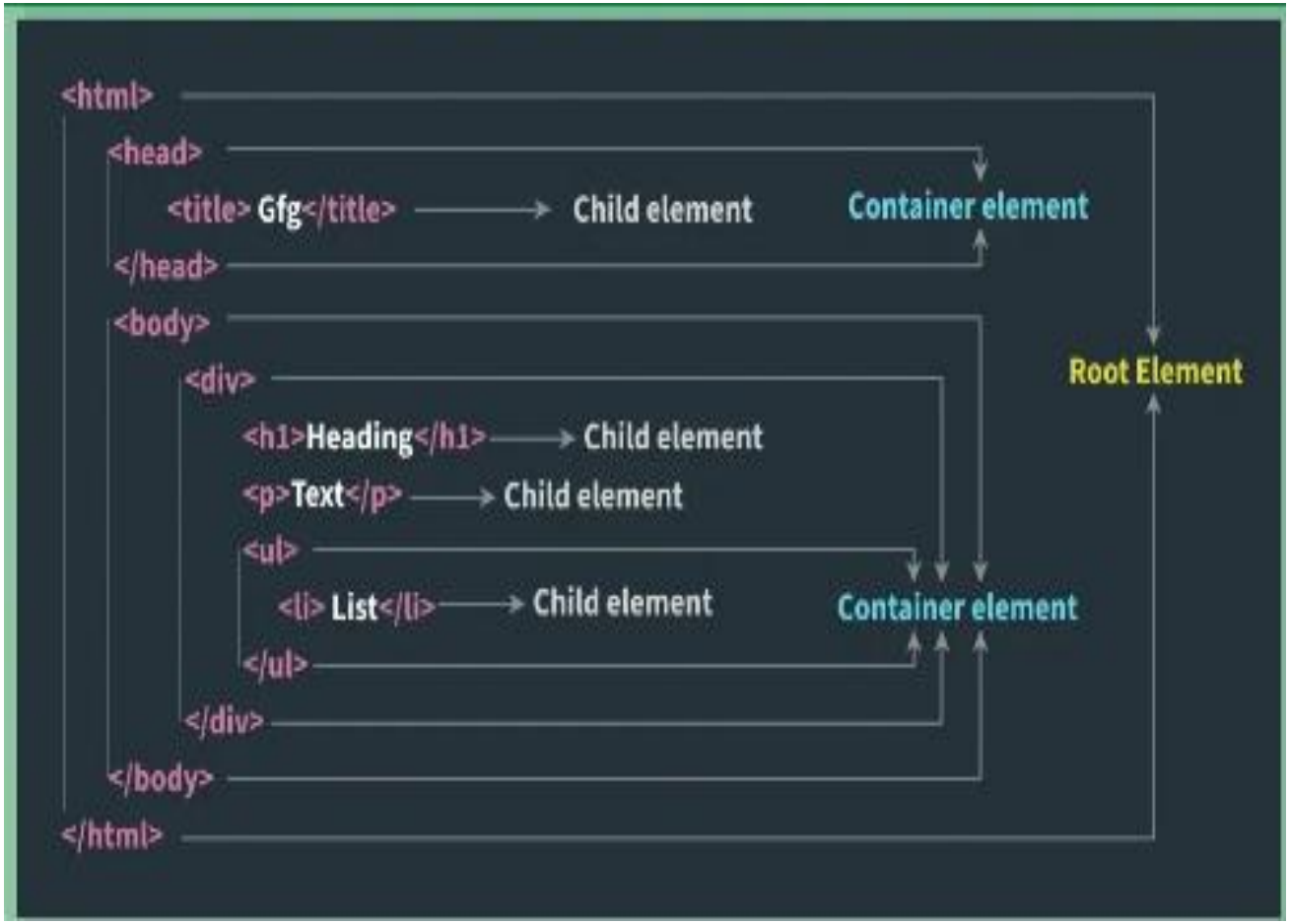
With the object model:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes


The "DOM Tree"
<html>
<head>
<title> Webpage Title</title>
</head>
<body>

<h1>This is Heading Tag</h1>
Some Text Content
<ul>
<li> List Item</li>
</ul>

</body>
</html>
Document
HTML
Head
Body
Title
div
p
h1
ul
li


<html>
<head>
<title> Gfg</title>
Child element
Container element
</head>
<body>
Root Element

<h1>Heading</h1>
Child element
Text
Child element
<ul>
<li> List</li>
Child element
Container element
</ul>

</body>
</html>

What Does the HTML DOM Look Like?

- The document is the root.
- HTML tags like <html>, <head>, and <body> are branches.
- Attributes, text, and other elements are the leaves.

JavaScript - HTML DOM Methods

- HTML DOM methods are **actions** you can perform (on HTML Elements).
- HTML DOM properties are **values** (of HTML Elements) that you can set or change.
-

The getElementById Method

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Page</h2>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```

- The most common way to access an HTML element is to use the id of the element.

In the example above the getElementById metho d used id="demo" to find the element.

The innerHTML Property

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Page</h2>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```

- The easiest way to get the content of an element is by using
the innerHTML property.

The innerHTML property is useful for getting or replacing the content of HTML elements.

Finding HTML Elements

Method	Description
document.getElementById(*id*)	Find an element by element id
document.getElementsByTagName(*name*)	Find elements by tag name
document.getElementsByClassName(*name*)	Find elements by class name

Changing HTML Elements

Property	Description
element.innerHTML = *new html content*	Change the inner HTML of an element
element.attribute = *new value*	Change the attribute value of an HTML element
element.style.*property* = *new style*	Change the style of an HTML element
Method	**Description**
element.setAttribute(*attribute, value*)	Change the attribute value of an HTML element

Adding and Deleting Elements

Method	Description
document.createElement(*element*)	Create an HTML element
document.removeChild(*element*)	Remove an HTML element
document.appendChild(*element*)	Add an HTML element
document.replaceChild(*new*, *old*)	Replace an HTML element
document.write(*text*)	Write into the HTML output stream

Adding Events Handlers

Method	Description
document.getElementById(*id*).onclick = function() {*code*}	Adding event handler code to an onclick event

DOM Levels

- (DOM Level 0 and Intermediate DOM)
  - Not W3C Standards, used in Netscape 2 (Level 0) and Netscape 4 (Intermediate DOM)
- DOM Level 1
  - 1998: Ability to change entire HTML or XML document
- DOM Level 2
  - 2001: Introduces “getElementById” function, event model and support for XML namespaces
- DOM Level 3
  - 2004: XPath, keyboard event handling

DOM Levels

- **Level 0:** Provides a low-level set of interfaces.
- **Level 1:** DOM level 1 can be described in two parts: **CORE** and **HTML**.
  - **CORE** provides low-level interfaces that can be used to represent any structured document.
  - **HTML** provides high-level interfaces that can be used to represent HTML documents.

DOM Levels

- **Level 2:** consists of six specifications: **CORE2**, **VIEWS**, **EVENTS**, **STYLE**, **TRAVER SAL**, and **RANGE**.

1. **CORE2**: extends the functionality of CORE specified by DOM level 1.
2. **VIEWS**: views allows programs to dynamically access and manipulate the content of the document.
3. **EVENTS**: Events are scripts that are either executed by the browser when the user reacts to the web page.
4. **STYLE**: allows programs to dynamically access and manipulate the content of style sheets.
5. **TRAVERSAL**: This allows programs to dynamically traverse the document.

DOM Levels

- **Level 3:** consists of five different specifications: **CORE3**, **LOAD and SAVE**, **VALIDATION**, **EVENTS**, and **XPATH**.

  1. **CORE3**: extends the functionality of CORE specified by DOM level 2.
  2. **LOAD and SAVE**: This allows the program to dynamically load the content of the XML document into the DOM document and save the DOM Document into an XML document by serialization.

Finding HTML Objects

Property	Description	DOM
document.anchors	Returns all <a> elements that have a name attribute	1
document.applets	Deprecated	1
document.baseURI	Returns the absolute base URI of the document	3
document.body	Returns the <body> element	1
document.cookie	Returns the document's cookie	1
document.doctype	Returns the document's doctype	3
document.documentElement	Returns the <html> element	3
document.documentMode	Returns the mode used by the browser	3
document.documentURI	Returns the URI of the document	3
document.domain	Returns the domain name of the document server	1
document.domConfig	Obsolete.	3